

Random Forest Classification – Airline Customer Satisfaction Analysis

Project: Predict passenger satisfaction using survey data from Invistico Airlines.

```
from google.colab import files
import pandas as pd

# Upload the CSV file
uploaded = files.upload()

# Load the dataset
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(f"✅ Dataset loaded: {df.shape}")
print(df.head())
print("\nTarget Distribution:")
print(df['satisfaction'].value_counts())
```

Choose Files Invistico_Airline.csv

Invistico_Airline.csv(text/csv) - 11960454 bytes, last modified: 6/22/2026 - 100% done
 Saving Invistico_Airline.csv to Invistico_Airline.csv
 ✅ Dataset loaded: (129880, 22)

	satisfaction	Customer Type	Age	Type of Travel	Class	\
0	satisfied	Loyal Customer	65	Personal Travel	Eco	
1	satisfied	Loyal Customer	47	Personal Travel	Business	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	

	Flight Distance	Seat comfort	Departure/Arrival time convenient	\
0	265	0	0	
1	2464	0	0	
2	2138	0	0	
3	623	0	0	
4	354	0	0	

	Food and drink	Gate location	...	Online support	Ease of Online booking	\
0	0	2	...	2	3	
1	0	3	...	2	3	
2	0	3	...	2	2	
3	0	3	...	3	1	
4	0	3	...	4	2	

	On-board service	Leg room service	Baggage handling	Checkin service	\
0	3	0	3	5	
1	4	4	4	2	
2	3	3	4	4	
3	1	0	1	4	
4	2	0	2	4	

	Cleanliness	Online boarding	Departure Delay in Minutes	\
0	3	2	0	
1	3	2	310	
2	4	2	0	
3	1	3	0	
4	2	5	0	

	Arrival Delay in Minutes
0	0.0
1	305.0
2	0.0
3	0.0
4	0.0

[5 rows x 22 columns]

Target Distribution:

```
satisfaction
satisfied      71087
dissatisfied   58793
Name: count, dtype: int64
```

```
# Handle missing values
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].median())

# One-hot encode categorical features
```

```
df = pd.get_dummies(df, drop_first=True)

# Define features and target
X = df.drop('satisfaction_satisfied', axis=1)
y = df['satisfaction_satisfied']

print(f"Features after encoding: {X.shape[1]}")
print("✅ Preprocessing completed")
```

Features after encoding: 22
✅ Preprocessing completed

```
from sklearn.model_selection import train_test_split

# 60% Train, 20% Validation, 20% Test
X_train_val, X_test, y_train_val, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.25, random_state=42)

print(f"Train set: {X_train.shape}")
print(f"Validation set: {X_val.shape}")
print(f"Test set: {X_test.shape}")
```

Train set: (77928, 22)
Validation set: (25976, 22)
Test set: (25976, 22)

```
from sklearn.model_selection import PredefinedSplit, GridSearchCV
from sklearn.ensemble import RandomForestClassifier

# Create PredefinedSplit
split_index = [-1] * len(X_train) + [0] * len(X_val)
ps = PredefinedSplit(split_index)

# Parameter grid
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [15, 20, None],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

print("✅ Ready for GridSearchCV")
```

✅ Ready for GridSearchCV

```
# Grid Search
rf = RandomForestClassifier(random_state=42, n_jobs=-1)

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring='f1',
    cv=ps,
    n_jobs=-1,
    verbose=1
)

# Fit on train + val
grid_search.fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val]))

print("✅ Best Parameters:", grid_search.best_params_)
print("Best F1 Score (CV):", round(grid_search.best_score_, 4))
```

Fitting 1 folds for each of 36 candidates, totalling 36 fits
✅ Best Parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'n_estimators': 200}
Best F1 Score (CV): 0.956

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

best_rf = grid_search.best_estimator_
y_pred = best_rf.predict(X_test)

print("=== Random Forest Test Performance ===")
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall   :", round(recall_score(y_test, y_pred), 4))
```

```
print("F1 Score :", round(f1_score(y_test, y_pred), 4))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
=== Random Forest Test Performance ===
Accuracy : 0.9567
Precision: 0.9694
Recall    : 0.9513
F1 Score  : 0.9603
```

```
Classification Report:
              precision    recall  f1-score   support

   False      0.94      0.96      0.95     11675
    True      0.97      0.95      0.96     14301

 accuracy          0.96          0.96          0.96     25976
  macro avg       0.96          0.96          0.96     25976
 weighted avg     0.96          0.96          0.96     25976
```

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=10, random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

print("=== Decision Tree Test Performance ===")
print("Accuracy :", round(accuracy_score(y_test, y_pred_dt), 4))
print("F1 Score :", round(f1_score(y_test, y_pred_dt), 4))
```

```
=== Decision Tree Test Performance ===
Accuracy : 0.9237
F1 Score : 0.9298
```

```
import matplotlib.pyplot as plt
import seaborn as sns

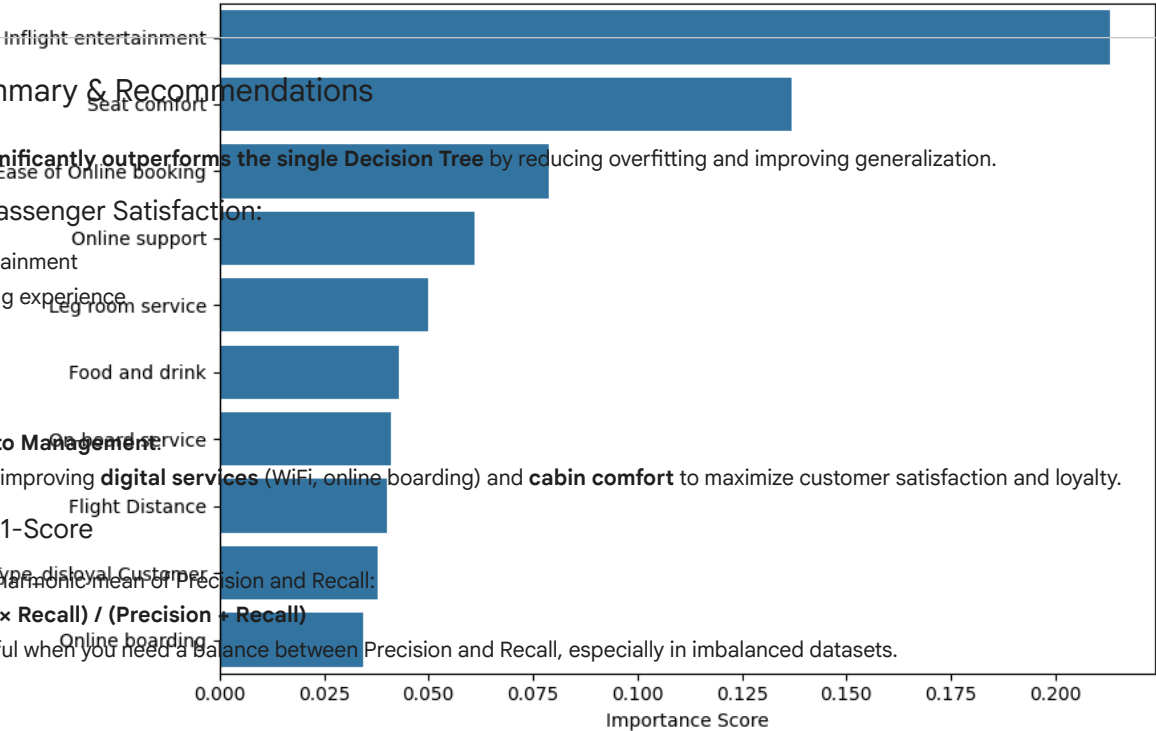
importances = pd.Series(best_rf.feature_importances_, index=X.columns).sort_values(ascending=False)

print("Top 10 Most Important Features:")
print(importances.head(10))

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(x=importances.head(10).values, y=importances.head(10).index)
plt.title('Top 10 Feature Importances - Random Forest')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()
```

```
Top 10 Most Important Features:
Inflight entertainment      0.213191
Seat comfort                0.136907
Ease of Online booking     0.078805
Online support              0.060858
Leg room service            0.049870
Food and drink              0.043004
On-board service            0.041154
Flight Distance             0.039913
Customer Type_disloyal Customer 0.037982
Online boarding             0.034274
dtype: float64
```

Top 10 Feature Importances - Random Forest



Executive Summary & Recommendations

Random Forest significantly outperforms the single Decision Tree by reducing overfitting and improving generalization.

Key Drivers of Passenger Satisfaction:

- In-flight entertainment
- Online boarding experience
- Seat comfort
- WiFi service
- Cleanliness

Recommendation to Management:

Focus resources on improving digital services (WiFi, online boarding) and cabin comfort to maximize customer satisfaction and loyalty.

Explanation of F1-Score

The F1-score is the Harmonic mean of Precision and Recall:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

It is particularly useful when you need a balance between Precision and Recall, especially in imbalanced datasets.